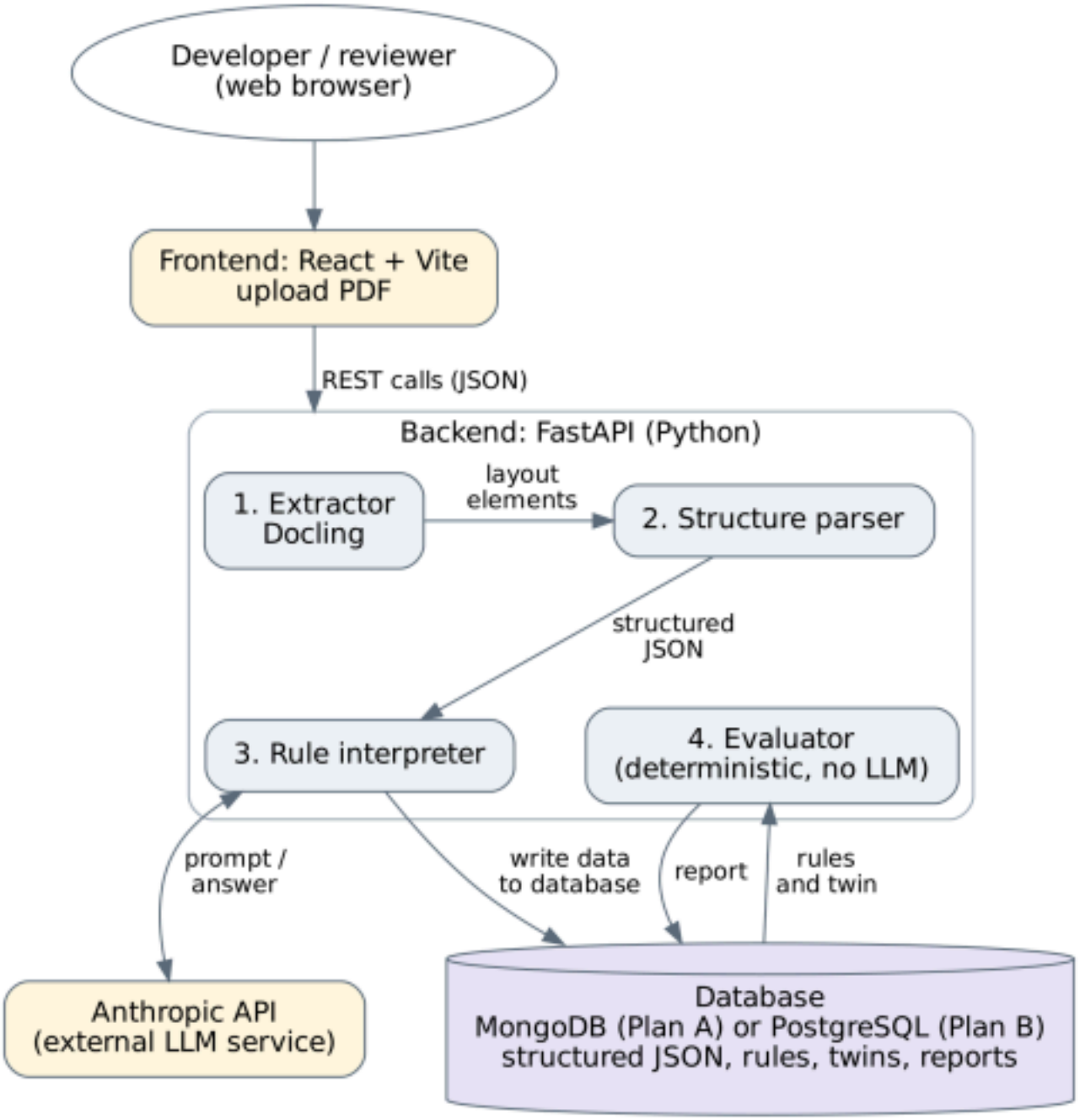
1. Runtime components

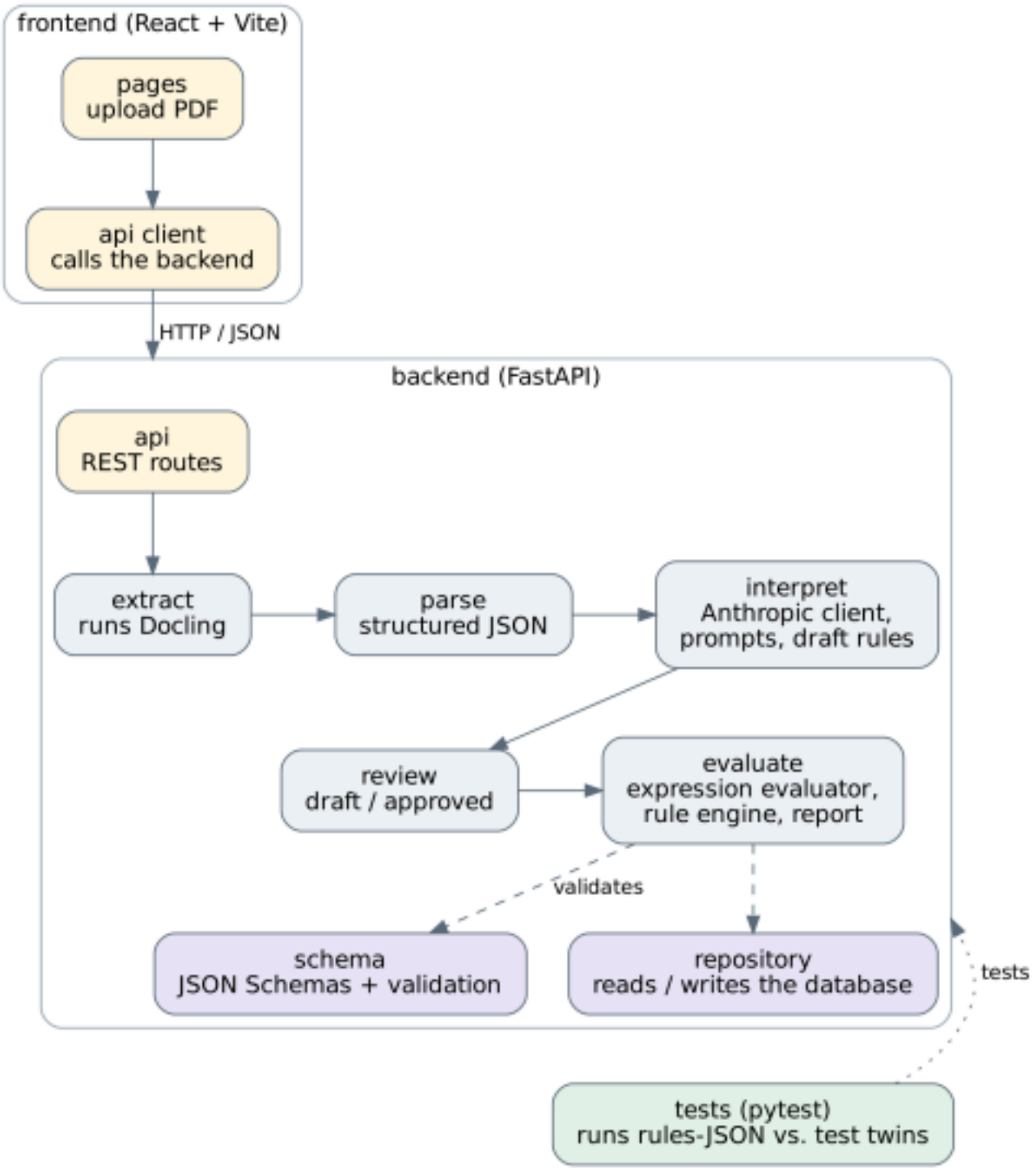
Phase 1 turns a PDF into rules. Phase 2 checks a digital twin against those rules.



Yellow: user interface or external service. Grey: our backend components. Purple: stored data.

2. Code components

Two code parts: a React frontend and a FastAPI backend. The backend has one module per pipeline step, and all modules share the JSON schemas. Runs rules-JSON vs test twins



Solid arrows: calls and data flow. Dashed: used by. Dotted: tested by.

3. Technology stack

The same list as in our software bill of materials. Rule data is stored as JSON and described with JSON Schema. Source code and tasks are kept on GitHub.

Context	Technology	Version	License	Why / status
Development / runtime	**Python**	3.14	PSF-2.0	Language of the backend; Docling and the Anthropic client are used from Python.
Document extraction	**Docling**	2.143.1	MIT	Extracts headings and paragraphs from the PDF.
LLM integration	**Anthropic**	2.3.0	MIT	Drafts rules from the structured JSON.
Backend framework	**FastAPI**	0.162.0	MIT	REST interface between the frontend and the pipeline.
Backend testing	**Pytest**	9.1.1	MIT	Tests rule JSON vs. test twins.
Frontend UI	**React**	19.3.1	MIT	Upload page for PDF files.
Frontend tooling	**Vite**	8.3.5	MIT	Build and development server for the frontend.
Database (Plan A)	**MongoDB**	9.0.4	SSPL-1.0	Stores JSON documents naturally. .
Database (Plan B)	**PostgreSQL**	18.6	PostgreSQL License	Relational database with JSON support. Fallback if Plan A is not suitable.

4. Explanation of the diagrams and choices

What the system does

ODOS needs to check digital twins against laws and standards. Laws exist as PDFs, so the system first turns a PDF into rules a computer can read (phase 1) and then compares a twin against those rules (phase 2). Our sources for now are the Musterbauordnung and the EU Machinery Regulation.

Runtime view (page 1)

A person uploads a PDF in the **React frontend**. The **FastAPI backend** runs four steps. The **extractor** (Docling) reads the PDF. The **structure parser** turns the text into JSON with section, paragraph, sentence and page. The **rule interpreter** asks the **Anthropic API** to draft rules with numbers, conditions and exceptions, and writes the data to the **database**. A person checks the rules before they are used. The **evaluator** reads the rules and a digital twin from the database and saves a report: pass, fail, needs review or not applicable, with the reason and the source sentence.

Code view (page 2)

The frontend has an upload page and a small client that calls the backend. In the backend, each step is its own module, so a step can be replaced without touching the others (for example Docling by another extractor, or our evaluator by pySHACL). The **schema** module checks every JSON file that moves between steps. The **repository** module is the only place that talks to the database, so Plan A and Plan B can be swapped.

Main choices and reasons

Two phases. The partner accepts that phase 1 is about 80% automatic with human review, but phase 2 must be fully deterministic. The LLM is therefore used only in phase 1, never in the check. **JSON.** The partner called it reasonable, and all developers know it.

Python backend and React frontend. Python fits Docling and the LLM client; a web page is an easy way to upload documents.

Human review. A person approves LLM output before it is used.

Open points

(1) Database: MongoDB (Plan A) or PostgreSQL (Plan B), including the MongoDB license. (2) Which format the ODOS ontology and twin use. (3) Whether Akoma Ntoso or pySHACL replace parts of the design after our research. (4) How well Docling works on both PDFs. We will update this document when these are decided.